

Yet Another TFG/TFM Handbook

Guía de trabajo para proyectos TFG/TFM en la ETSIIT - UGR

Juan Luis Jiménez Laredo

2025-01-01

Índice

1	 Introducción	1
1.1	A quién va dirigido este handbook	1
1.2	Cómo usar este handbook	2
1.3	Referencias y recursos complementarios	2
1.3.1	Libro sobre TFG/TFM en la UGR	3
1.3.2	Plantilla de TFG de JJ Merelo (ETSIIT)	3
1.4	Filosofía de trabajo	3
1.5	TFG/TFM dirigidos (ejemplos)	4
1.6	Licencia	4
2	 Metodología ágil para TFG/TFM	5
2.1	Objetivo de este capítulo	5
2.2	Enfoque general: ágil adaptado a TFG/TFM	5
2.3	Roles en el proyecto	5
2.3.1	Alumno/a (equipo de desarrollo)	6
2.3.2	Director/a y/o alumno (Product Owner)	6
2.3.3	Otros actores (stakeholders)	6
2.4	Artefactos y herramientas	6
2.4.1	Backlog de producto	7
2.4.2	Milestones (hitos)	7
2.4.3	Tablero (Project) en GitHub	7
2.5	Sprints: cómo iterar el trabajo	8
2.5.1	Duración	8
2.5.2	Estructura de un sprint	8
2.5.3	Ejemplo de sprint típico	8
2.6	Fases típicas de un TFG/TFM (vista ágil)	9
2.6.1	Fase 1: Arranque y planificación	9
2.6.2	Fase 2: Diseño / preparación	9
2.6.3	Fase 3: Implementación iterativa	9
2.6.4	Fase 4: Evaluación y refinamiento	9
2.6.5	Fase 5: Cierre y documentación	9
2.7	Calidad, pruebas y CI (opcional para el alumno)	10
2.7.1	Calidad mínima esperada	10
2.7.2	Integración continua (CI) como opción	10
2.8	Expectativas de comunicación	10
2.9	Cómo se concreta esto en tu TFG/TFM	11
3	GitHub y organización del proyecto	13
3.1	Objetivo de este capítulo	13
3.2	Organización de GitHub por proyecto	13
3.3	Repositorios mínimos recomendados	14
3.4	Projects, Milestones e Issues	14
3.4.1	Milestones (hitos)	14
3.4.2	Issues (tareas)	15

3.4.3	Project (tablero Kanban)	15
3.5	Ramificación (branches) y flujo de trabajo básico	15
3.6	Integración continua (CI): recomendada, no obligatoria	16
3.6.1	¿Cuándo tiene sentido usar CI?	16
3.6.2	Ejemplo de workflow mínimo (Python)	16
3.7	Checklist inicial para el alumno	17
4	Memoria de TFG/TFM: estructura y recomendaciones	19
4.1	Objetivo de este capítulo	19
4.2	Estructura recomendada de la memoria	19
4.3	Introducción	20
4.4	Contexto y estado del arte	20
4.5	Objetivos y alcance	21
4.5.1	Objetivo general	21
4.5.2	Objetivos específicos	21
4.5.3	Alcance y limitaciones	21
4.6	Metodología	21
4.7	Análisis y requisitos	22
4.8	Diseño y arquitectura	22
4.9	Implementación	23
4.10	Evaluación / experimentación	23
4.11	Conclusiones y trabajo futuro	24
4.12	Referencias	24
4.13	Anexos	25
4.14	Recomendaciones prácticas	25
4.15	Recursos complementarios recomendados	25
4.15.1	Libro sobre TFG/TFM en la ETSIIT	25
4.15.2	Plantilla de TFG de JJ Merelo (ETSIIT)	26

1 Introducción

Versión web

Este texto forma parte de la versión en formato **libro** / **PDF** del handbook. Existe también **una versión web** que presenta el mismo contenido de forma navegable.

Este libro recoge las **reglas del juego** para los TFG/TFM que dirijo en la ETSIIT de la Universidad de Granada.

No pretende ser **la** guía definitiva, sino “**otro manual más**” (*yet another handbook*) que:

- Explicita cómo suelo trabajar con mis alumnos.
- Complementa otros recursos existentes en la UGR y en la ETSIIT.
- Sirve como referencia viva y versionada para los proyectos que dirijo.

Aquí encontrarás:

- Cómo organizaremos tu TFG/TFM usando **metodologías ágiles** (sprints, backlog, issues...).
- Cómo usaremos **GitHub**:
 - Creación de una **organización por proyecto** (con un nombre en clave, p.ej. TempusUGR).
 - Repositorios de código y documentación.
 - Projects, Issues y Milestones para gestionar el trabajo.
- Qué se espera de la **memoria del TFG/TFM** (estructura, contenido mínimo, calidad).

Importante

Este handbook **no es la guía oficial** de TFG/TFM de la ETSIIT ni de la UGR, ni sustituye a la normativa vigente del centro.

Es simplemente “*otro handbook más*” que documenta **cómo suelo trabajar con mis alumnos** y ofrece un marco práctico de referencia.

1.1. A quién va dirigido este handbook

Este material está pensado principalmente para:

- Estudiantes de la ETSIIT que van a realizar un **TFG o TFM** conmigo como director (o co-director).
- Estudiantes que quieran ver **un ejemplo concreto** de cómo estructurar un proyecto de fin de estudios en Ingeniería Informática usando GitHub y metodologías ágiles.

- Cualquier persona interesada en:
 - Aplicar enfoques ágiles en proyectos individuales.
 - Organizar proyectos en GitHub de forma ordenada.
 - Entender qué se espera de la **memoria** de un TFG/TFM de ingeniería.

En caso de duda o conflicto, la **normativa oficial** de la UGR y de la ETSIIT siempre tiene prioridad sobre lo que se describe aquí.

1.2. Cómo usar este handbook

Si vas a hacer un TFG/TFM conmigo, el flujo recomendado es:

1. Leer la sección Metodología ágil para entender cómo organizaremos el trabajo en sprints.
2. Revisar GitHub & organización para crear la organización de tu proyecto y los repositorios mínimos.
3. Ver Memoria TFG/TFM para saber qué estructura se espera en la memoria.
4. Consultar, cuando esté disponible, el **documento de trabajo específico** de tu proyecto (por ejemplo, el del TFG *Pastillero digital inteligente*).

Primeros pasos recomendados

1. Crea la **organización de GitHub** de tu proyecto con un nombre en clave (por ejemplo, PastilleroUGR).
2. Crea al menos:
 - Un repositorio para el **código**.
 - Un repositorio para la **documentación/memoria**.
3. Configura un **Project** con columnas tipo Backlog, En curso, Hecho y crea los primeros *issues* de tareas.
4. Abre y revisa regularmente tu **documento de trabajo específico** para mantener alineadas las expectativas con el director.

1.3. Referencias y recursos complementarios

Este handbook **no sustituye** a otros materiales ya existentes en la UGR y en la ETSIIT, sino que los **complementa**.

Es muy recomendable que, además de leer estas páginas, revises:

1.3.1. Libro sobre TFG/TFM en la UGR

- Disponible en la producción científica de la UGR:
<https://produccioncientifica.ugr.es/documentos/67e6f8553f806d20a93a1b2a>

Este libro ofrece una visión más amplia sobre:

- Qué es un TFG/TFM en el contexto de la UGR.
- Recomendaciones generales sobre redacción académica.
- Aspectos formales, evaluación, etc.

1.3.2. Plantilla de TFG de JJ Merelo (ETSIIT)

- Repositorio: <https://github.com/JJ/plantilla-TFG-ETSIIT>

Esta plantilla es especialmente útil si:

- Quieres escribir la memoria en **LaTeX**.
- Prefieres partir de una estructura ya muy pulida y probada para TFG de la ETSIIT.
- Te interesa ver un ejemplo de cómo automatizar parte del proceso de compilación.

Idea práctica

Puedes usar este handbook para la **forma de trabajar** (metodología, GitHub, planificación) y, al mismo tiempo, usar la **plantilla de JJ** o el libro de la UGR como referencia para el estilo y formato de tu memoria.

1.4. Filosofía de trabajo

1. Transparencia

Todo el trabajo relevante (código, documentación, tareas) vive en GitHub.

2. Iteración

Trabajamos con sprints cortos y entregas incrementales, evitando dejarlo todo para el final.

3. Calidad y reflexión

El objetivo no es solo “que funcione”, sino también:

- Que la solución tenga sentido en su contexto.
- Que el código sea razonablemente limpio y mantenible.
- Que la memoria cuente una historia coherente del proyecto.

4. Adaptación al tipo de proyecto

No todos los proyectos son iguales:

- En algunos habrá más peso de **desarrollo de software**.
- En otros, más de **experimentación, análisis de datos o investigación**. Las pautas de este handbook se adaptan a esa diversidad.

1.5. TFG/TFM dirigidos (ejemplos)

En la versión web de este handbook se incluye una sección de **TFG/TFM dirigidos**, donde se irán enlazando los proyectos concretos que superviso, por ejemplo:

- 2024–2025 · TempusUGR

Cada uno incluye un resumen, enlaces a su organización de GitHub y, cuando procede, su documento de trabajo específico.

En el contexto del libro en PDF puedes usar esa sección como referencia cruzada para consultar ejemplos reales en la versión web.

1.6. Licencia

Este handbook contiene principalmente documentación y algo de código de ejemplo.

- El **contenido textual y la documentación** (ficheros `.qmd`, `.md`, descripciones, etc.) se publican bajo la licencia **Creative Commons Atribución 4.0 Internacional (CC BY 4.0)**.
- El **código y las configuraciones** (por ejemplo, workflows de GitHub Actions y scripts) se publican bajo la licencia **MIT**.

En la práctica, esto significa que puedes reutilizar, adaptar y compartir tanto la documentación como el código, incluso en contextos diferentes a la ETSIIT, siempre que:

- Des el crédito adecuado al autor y a este repositorio.
- Mantengas el aviso de licencia correspondiente (CC BY para el contenido, MIT para el código).
- Indiques si has realizado modificaciones cuando corresponda.

Para más detalles, consulta el archivo de licencia incluido en el repositorio original del que se genera este libro <https://github.com/juanluck/yet-another-tfg-tfm-handbook>.

2 ≡ Metodología ágil para TFG/TFM

2.1. Objetivo de este capítulo

Este capítulo explica **cómo trabajaremos tu TFG/TFM usando metodologías ágiles** (Scrum/Kanban ligeros) en el contexto de la ETSIIT (UGR).

No define la metodología concreta de tu proyecto (eso irá en tu **documento de trabajo específico**), sino el **marco general**:

- Cómo organizamos el trabajo en **iteraciones**.
 - Qué **artefactos** esperamos (backlog, issues, milestones...).
 - Cómo usamos **GitHub** para gestionar el proyecto.
 - Qué se espera de ti antes, durante y después de cada sprint.
-

2.2. Enfoque general: ágil adaptado a TFG/TFM

En un TFG/TFM no tenemos un equipo grande, sino:

- Una persona que **desarrolla** (tú, el alumno).
- Una o varias personas que **dirigen/guían** (director/a, co-directores).

Por tanto, usamos una versión **reducida y pragmática** de metodologías ágiles:

- Trabajamos en **sprints cortos** (1–2 semanas).
- Mantenemos un **backlog** de tareas (issues en GitHub).
- Planificamos y revisamos el trabajo de forma **iterativa**.
- El objetivo de cada sprint es tener algo **enseñable** (prototipo, módulo, experimento, capítulo de la memoria, etc.).

No buscamos cumplir Scrum “al pie de la letra”, sino quedarnos con lo que aporta valor en un TFG/TFM.

2.3. Roles en el proyecto

En este contexto, los roles se mapean así:

2.3.1. Alumno/a (equipo de desarrollo)

- Eres responsable de:
 - Planificar tu trabajo por sprints.
 - Mantener el **backlog** y actualizar el estado de las tareas.
 - Desarrollar código, realizar experimentos, escribir documentación.
 - Preparar las demos/entregables de cada sprint.
 - También asume parte del rol de **Product Owner**

2.3.2. Director/a y/o alumno (Product Owner)

- Actúa como una especie de **Product Owner**:
 - Ayuda a definir y priorizar objetivos.
 - Da feedback sobre lo entregado.
 - Valida el alcance del proyecto y si se cumplen los objetivos del TFG/TFM.

2.3.3. Otros actores (stakeholders)

Según el proyecto, puede haber:

- Usuarios finales.
- Otros profesores o grupos de investigación.
- Colaboradores externos (empresas, ayuntamientos, etc.).

Su presencia e implicación se definirá en tu documento de trabajo específico.

2.4. Artefactos y herramientas

Trabajaremos principalmente con:

- **Organización de GitHub** específica de tu proyecto.
- Uno o varios **repositorios** (código, documentación...).
- Un **Project** de GitHub (tablero Kanban).
- **Issues** y **milestones** para gestionar tareas e hitos.

Los detalles prácticos (cómo crear la organización, repos, Projects, etc.) están en GitHub & organización.

2.4.1. Backlog de producto

El **backlog** es la lista de todo lo que vamos a hacer:

- Historias de usuario (si tu proyecto las usa).
- Tareas técnicas (configurar CI, escribir tests, refactorizar código, etc.).
- Tareas de documentación (escribir secciones de la memoria, preparar gráficos).
- Tareas de evaluación (diseñar experimentos, preparar cuestionarios, etc.).

En la práctica:

- Cada elemento del backlog es **un issue de GitHub**.
- Deberían tener etiquetas (**backend**, **frontend**, **documentation**, **research**, etc.).
- Se asignan a **milestones** (hitos) y se van moviendo en tu tablero Project.

2.4.2. Milestones (hitos)

Recomendamos definir milestones asociados a fases del TFG/TFM. Ejemplo típico:

- **M1 – Análisis y planificación inicial**
- **M2 – Diseño / arquitectura**
- **M3 – Implementación del MVP**
- **M4 – Evaluación / experimentación**
- **M5 – Cierre y memoria**

Cada milestone tiene:

- Una **fecha objetivo**.
- Un conjunto de issues que deben completarse para considerarlo cumplido.

2.4.3. Tablero (Project) en GitHub

Utilizaremos un **Project** (tablero Kanban) para visualizar el estado de las tareas.

- Columnas típicas:
 - Backlog
 - En curso
 - En revisión
 - Hecho

Cada issue se mueve de columna según su estado.

Esto ayuda a ver de un vistazo en qué estás trabajando y qué queda pendiente.

2.5. Sprints: cómo iterar el trabajo

2.5.1. Duración

En general:

- Sprints de **1 o 2 semanas** funcionan bien para un TFG/TFM.
- La duración exacta se acuerda al inicio del proyecto (y se puede ajustar).

2.5.2. Estructura de un sprint

Cada sprint debería incluir, como mínimo:

1. Planning (inicio de sprint)

- Revisar el backlog con el director (cuando sea posible).
- Elegir qué issues se abordarán en el sprint.
- Definir un **objetivo claro** para las próximas 1–2 semanas.

2. Ejecución

- Desarrollar código, experimentar, escribir memoria, etc.
- Crear issues si surgen nuevas tareas.
- Mantener el Project actualizado.

3. Revisión

- Preparar una **demo o resumen** de lo hecho (pantallazos, prototipo, resultados preliminares, sección de memoria redactada...).
- Comentar con el director:
 - Qué se ha conseguido.
 - Qué ha bloqueado o retrasado trabajo.

4. Retrospectiva informal

- Reflexionar brevemente:
 - ¿Qué ha ido bien?
 - ¿Qué no?
 - ¿Qué cambiarás en el siguiente sprint?

No hace falta que esto sea pesado: puede ser un mensaje estructurado o una breve reunión.

2.5.3. Ejemplo de sprint típico

Sprint (2 semanas)

Objetivo: tener un prototipo funcional de las pantallas principales.

Tareas (issues):

- Diseñar wireframes de la pantalla de inicio.
- Implementar la navegación básica entre pantallas.
- Crear la estructura básica del proyecto (repositorio, dependencias).
- Documentar las decisiones de diseño en el repositorio de documentación.

Entregable:

- Repositorio con app que navega entre pantallas básicas.
- Documento con capturas y explicación del flujo.

2.6. Fases típicas de un TFG/TFM (vista ágil)

Aunque trabajemos con sprints, normalmente podemos distinguir varias **fases**:

2.6.1. Fase 1: Arranque y planificación

- Lectura del enunciado y asignación de TFG/TFM.
- Revisión de documentación inicial y trabajo relacionado.
- Definición del **alcance preliminar** con el director.
- Creación de:
 - Organización de GitHub.
 - Repositorios iniciales.
 - Project y primeros milestones.
- Redacción y/o revisión del **documento de trabajo específico** del TFG/TFM.

2.6.2. Fase 2: Diseño / preparación

- Diseño de arquitectura (software, hardware o ambos).
- Diseño de modelos de datos, APIs, flujos, etc.
- Plan de experimentación/evaluación si aplica.
- Inicio de redacción de capítulos de la memoria (no esperes al final).

2.6.3. Fase 3: Implementación iterativa

- Desarrollo de un **MVP** (mínimo producto viable) lo antes posible.
- Iteraciones sobre funcionalidades, diseño, rendimiento, etc.
- Tests básicos (unitarios, de integración, manuales).

2.6.4. Fase 4: Evaluación y refinamiento

- Pruebas con usuarios, datasets, benchmarks... según el tipo de proyecto.
- Análisis de resultados.
- Refinamiento de la solución (ajustes de rendimiento, usabilidad, etc.).

2.6.5. Fase 5: Cierre y documentación

- Consolidación del código (limpieza, documentación interna).
- Redacción final de la memoria y anexos.
- Preparación de la defensa (presentación, demo, etc.).

Estas fases **no son lineales**: se solapan y se revisan, pero sirven como guía para tus milestones.

2.7. Calidad, pruebas y CI (opcional para el alumno)

2.7.1. Calidad mínima esperada

Independientemente de que uses CI o no, se espera:

- Código razonablemente estructurado, legible y documentado.
- Algún tipo de **pruebas**:
 - Unitarias y/o de integración (según el tipo de proyecto).
 - Pruebas manuales reproducibles (pasos claros).
- Un repositorio con una mínima organización:
 - `src/`, `tests/`, `docs/`, etc.
 - Un README que explique cómo ejecutar el proyecto.

2.7.2. Integración continua (CI) como opción

La **integración continua no es obligatoria**, pero se recomienda cuando:

- El proyecto tiene cierto tamaño/ complejidad.
- Se pueden automatizar:
 - Ejecución de tests.
 - Análisis estático (linters).
 - Generación de documentación.

Si decides usar CI:

- Lo habitual es usar **GitHub Actions**.
- Configurar al menos un workflow que:
 - Se ejecute en cada push o pull request.
 - Lance tests y/o linters.

En el handbook (repositorio del propio handbook) sí se usa CI para generar la web, y puedes tomarlo como **ejemplo**, pero **no es requisito** que repliques exactamente esa configuración en tu proyecto.

2.8. Expectativas de comunicación

Para que la metodología ágil funcione mínimamente, es importante:

- Acotar una **frecuencia de contacto** razonable con el director (por ejemplo, cada 2–3 semanas).
- Antes de cada reunión, preparar:
 - Qué has hecho (issues cerrados, demos).
 - Qué problemas has encontrado.
 - Qué te propones hacer en el siguiente periodo.

Una buena práctica es:

- Abrir un issue tipo “Informe de sprint N” donde:
 - Resumas los avances.
 - Enlaces a commits, PRs, capturas de pantalla, etc.
 - Anotes dudas para la reunión.
-

2.9. Cómo se concreta esto en tu TFG/TFM

Todo lo anterior es el **marco general**.

Para tu proyecto concreto:

- Recibirás un **documento de trabajo** (en este mismo handbook), por ejemplo:
 - `cuentagotas-doc-trabajo.qmd` para un TFG que desarrolla un cuenta gotas.
- Ese documento:
 - Detalla la metodología específica (por ejemplo, si se usa DCU, técnicas concretas de evaluación, etc.).
 - Aterrizas los requisitos, la arquitectura y el planning de tu proyecto.
 - Se irá actualizando si es necesario (versionado en Git).

Siempre que tengas dudas entre lo genérico (este documento) y lo específico de tu TFG/TFM, **prima lo que se indique en tu documento de trabajo concreto**, o consulta al director.

3 GitHub y organización del proyecto

3.1. Objetivo de este capítulo

Este capítulo explica **cómo organizaremos el proyecto en GitHub**:

- Creación de una **organización por proyecto** (en lugar de usar tu cuenta personal).
- Estructura recomendada de **repositorios**.
- Uso de **Projects, Issues y Milestones** para gestionar el trabajo.
- Recomendaciones generales sobre **integración continua (CI)**.

La CI se propone como opción recomendada, no obligatoria: tú decides si la aplicas en tu TFG/TFM.

En este handbook sí se usa CI para generar la web, y puedes tomarlo como ejemplo. Puedes echar un vistazo a su repositorio de github.

3.2. Organización de GitHub por proyecto

En la mayoría de TFG/TFM que dirija seguiremos esta idea:

- Crear una **organización específica en GitHub** para el proyecto.
- El nombre será un **nombre en clave**, no tu nombre personal.

Ejemplo de organización de TFG ya creada:

- TempusUGR

Ejemplos de nombres de organización:

- PastilleroUGR
- HorarioUGR
- SenseUGR

Ventajas de usar organizaciones:

- Puedes agrupar varios repositorios (código, documentación, infraestructura...).
- Es más fácil colaborar con otras personas (añadir miembros con distintos permisos).
- El resultado final queda más limpio y “profesional” que un repo suelto en una cuenta personal.

3.3. Repositorios mínimos recomendados

Dentro de la organización de tu proyecto, lo normal será tener al menos:

1. Repositorio de código principal

Ejemplos:

- `pastillero-app`
- `tempus-frontend`, `tempus-backend`

Estructura típica:

```
src/           # Código fuente
tests/         # Tests automatizados (si aplica)
docs/          # Documentación específica de ese repo (si la hay)
.github/
  workflows/   # Workflows de CI (opcional)
README.md
```

2. Repositorio de documentación/memoria

Por ejemplo:

- `pastillero-docs`
- `tempus-docs`

Aquí puedes guardar:

- Memoria del TFG/TFM (en LaTeX, Quarto, etc.).
- Anexos.
- Diagramas, figuras, datos, etc.

En proyectos más grandes se pueden añadir más repositorios (microservicios, librerías comunes, etc.), pero para un TFG/TFM normal con estos dos suele ser suficiente.

3.4. Projects, Milestones e Issues

3.4.1. Milestones (hitos)

Usa **milestones** para agrupar issues según fases del TFG/TFM.

Ejemplo de milestones:

- M1 - Análisis y planificación
- M2 - Diseño / arquitectura
- M3 - Implementación MVP
- M4 - Evaluación
- M5 - Cierre y memoria

Cada milestone debe tener:

- Una **fecha objetivo** razonable.
- Un conjunto de issues asignados.

3.4.2. Issues (tareas)

Cada tarea importante del proyecto debe ser un **issue**:

- Usa títulos claros, por ejemplo:
 - Definir historias de usuario iniciales
 - Diseñar modelo de datos
 - Implementar pantalla de toma de medicación
 - Escribir sección 3. Metodología de la memoria
- Añade una descripción breve:
 - Qué se espera conseguir.
 - Criterios de aceptación (cuando tenga sentido).
- Etiquetas recomendadas:
 - backend, frontend, documentation, research, ui-ux, bug, enhancement, etc.
- Asigna el issue a:
 - Un milestone.
 - Opcionalmente, a ti mismo (siempre serás el responsable principal).

3.4.3. Project (tablero Kanban)

Crea un **Project** de GitHub (board) asociado a la organización o a un repo principal.

Configuración mínima:

- Columnas:
 - Backlog
 - En curso
 - En revisión
 - Hecho

Cada issue se añade al Project y se mueve de columna según su estado.

Esto permite ver de un vistazo:

- Qué tareas hay pendientes.
- Qué estás haciendo ahora.
- Qué está ya terminado.

3.5. Ramificación (branches) y flujo de trabajo básico

No vamos a imponer un flujo Git muy complejo, pero sí se recomiendan unas pautas mínimas:

- Mantener una rama principal (**main** o **master**) **siempre en estado estable**.
- Para cambios medianos/grandes:
 - Crear una rama de feature, por ejemplo:

- `feature/pantalla-tomas`
 - `feature/modelo-datos`
 - Hacer commits en esa rama.
 - Crear un **pull request (PR)** a `main` cuando esté listo.
 - Si trabajas solo, puedes auto-mergear el PR, pero:
 - El PR sirve como “paquete” de cambios.
 - Puedes usar la descripción del PR para documentar qué has hecho.
-

3.6. Integración continua (CI): recomendada, no obligatoria

3.6.1. ¿Cuándo tiene sentido usar CI?

Tiene sentido plantearse CI cuando:

- Tienes una batería de **tests automatizados** (unitarios, de integración, etc.).
- El proyecto va a crecer o quieres mantener cierta disciplina técnica.
- Quieres automatizar tareas repetitivas:
 - Ejecutar tests en cada push.
 - Generar informes.
 - (Opcional) Generar builds o documentación.

No es obligatorio para el TFG/TFM, pero:

- Si lo usas bien, **suma puntos** a nivel de calidad y profesionalidad.
- Puedes tomar como referencia el workflow que usamos en este handbook (`quarto-handbook.yml`).

3.6.2. Ejemplo de workflow mínimo (Python)

Este es solo un ejemplo genérico para proyectos en Python:

```
# .github/workflows/tests.yml
name: Run tests

on:
  push:
    branches: [ main ]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4
```

```

- name: Set up Python
  uses: actions/setup-python@v5
  with:
    python-version: "3.11"

- name: Install dependencies
  run: |
    pip install -r requirements.txt

- name: Run tests
  run: |
    pytest

```

Adáptalo a:

- Node/TypeScript, Java, etc., según el stack de tu proyecto.
- Solo si realmente vas a usar tests.

3.7. Checklist inicial para el alumno

Cuando empieces tu TFG/TFM, deberías:

1. Crear la **organización** de GitHub con un nombre en clave del proyecto.
2. Crear al menos:
 - ☐ Un repositorio para el código.
 - ☐ Un repositorio para la documentación/memoria.
3. Configurar un **Project** (tablero) y los primeros **milestones**.
4. Crear los primeros issues de backlog (a partir del documento de trabajo y de las conversaciones con el director).
5. Documentar en el README del repositorio principal:
 - Objetivo del proyecto.
 - Cómo ejecutar el código.
 - Tecnologías principales.

La integración continua (CI) es recomendable pero opcional:

6. [] Si decides usarla, crea un workflow sencillo para lanzar tests o validaciones básicas en cada push.

4 Memoria de TFG/TFM: estructura y recomendaciones

4.1. Objetivo de este capítulo

Este capítulo describe la **estructura recomendada** para la memoria de tu TFG/TFM y algunas pautas de calidad.

No sustituye a la normativa oficial de la ETSIIT o de la UGR (que siempre tiene prioridad), pero:

- Te da una estructura base razonable para la mayoría de proyectos de Ingeniería Informática.
- Te ayuda a no dejar la memoria para el último momento: la idea es que se vaya **escribiendo en paralelo** al desarrollo.

Documentos oficiales

Puedes encontrar las plantillas oficiales de la ETSIIT en distintos formatos en esta URL. Como ejemplo de una buena memoria de TFG, puedes acceder al repositorio de documentación del TFG TempusUGR

4.2. Estructura recomendada de la memoria

La estructura concreta puede variar según el proyecto, pero una plantilla razonable es:

1. **Introducción**
2. **Contexto y estado del arte**
3. **Objetivos y alcance del proyecto**
4. **Metodología**
5. **Análisis y requisitos**
6. **Diseño (y arquitectura)**
7. **Implementación**
8. **Evaluación / Experimentación**
9. **Conclusiones y trabajo futuro**
10. **Referencias**
11. **Anexos**

En los apartados siguientes se detalla qué se espera en cada sección.

4.3. Introducción

La introducción debe responder, en pocas páginas, a:

- ¿Cuál es el **problema** que aborda el proyecto?
- ¿Por qué es **relevante**?
- ¿Cuál es la **situación previa** (muy resumida)?
- ¿Qué se pretende conseguir con este TFG/TFM?

Elementos recomendados:

- Breve contexto (sin todavía entrar a fondo en el estado del arte).
 - Objetivo general del trabajo (una frase clara).
 - Estructura de la memoria (describir brevemente qué contiene cada capítulo).
-

4.4. Contexto y estado del arte

Aquí debes:

- Analizar el contexto técnico y/o social del problema.
- Revisar trabajos relacionados, herramientas, librerías, sistemas similares, etc.

Recomendaciones:

- Agrupa los trabajos por temática (no hagas una lista sin hilo conductor).
- Compara y discute:
 - En qué se parecen a tu propuesta.
 - En qué se diferencian.
 - Qué hueco deja libre el estado del arte que tu TFG/TFM intenta cubrir.

En proyectos basados en software:

- Puedes analizar soluciones comerciales, proyectos open source, frameworks, etc.

En proyectos más “experimentales”:

- Puedes centrarse en técnicas, algoritmos, metodologías previas.

i Más allá del estado del arte

El último párrafo (o últimos párrafos) de un estado del arte deben responder a la cuestión: **¿en que mejoramos el estado del arte con este proyecto?**

La lógica es algo así como: *Muy bien, hasta ahora te he estado contando en esta sección todo los proyectos, referencias, frameworks, etc. que son relevantes para nuestro proyecto, pero ninguno de ellos aborda exactamente la problemática que estamos abordando (y, en caso de que si la aborde, tiene los defectos A, B o C). Por lo tanto, este TFG proporciona en este marco una solución **más allá del estado del arte** que cumple con los objetivos previstos y soluciona A, B o C.*

4.5. Objetivos y alcance

Aunque hayas mencionado los objetivos en la introducción, aquí se formalizan:

4.5.1. Objetivo general

- Una frase que describa el propósito principal del TFG/TFM.

4.5.2. Objetivos específicos

- Lista numerada de objetivos concretos (3–7 es un rango razonable).

Ejemplo genérico:

- Diseñar e implementar una aplicación que...
- Evaluar el rendimiento / precisión / usabilidad de...
- Integrar X y Y para demostrar que...

4.5.3. Alcance y limitaciones

- ¿Qué **sí** se va a hacer?
 - ¿Qué **no** entra en el alcance (límites del TFG/TFM)?
 - Justificar las decisiones (tiempo, complejidad, etc.).
-

4.6. Metodología

En este capítulo se explica **cómo** se ha trabajado:

- Metodologías de desarrollo (por ejemplo, enfoque ágil).
- Metodologías de investigación o experimentación (si aplica).
- En proyectos con personas usuarias:
 - Cómo se han realizado entrevistas, tests de usabilidad, encuestas, etc.

Recomendaciones:

- No basta con mencionar “Scrum” o “Kanban”: describe **cómo lo has adaptado** a tu TFG/TFM.
- Incluye:
 - Cómo has organizado los sprints.
 - Qué artefactos has usado (backlog, Project, issues).
 - Cómo se han planificado y revisado los hitos.

Si tu proyecto utiliza técnicas específicas (p.ej. DCU, experimentos controlados, etc.), descríbelas aquí o en subapartados.

4.7. Análisis y requisitos

En esta sección se pasa del problema general a requisitos concretos.

Contenido típico:

- Perfiles de usuario y casos de uso (cuando tiene sentido).
- Historias de usuario (si se usa este enfoque).
- Requisitos funcionales y no funcionales:
 - Funcionales: qué debe hacer el sistema.
 - No funcionales: rendimiento, seguridad, usabilidad, escalabilidad, etc.

Recomendaciones:

- No te limites a una lista plana: organiza por áreas funcionales.
 - Asegúrate de que los requisitos estén relacionados con los objetivos del proyecto.
-

4.8. Diseño y arquitectura

Aquí se describe **cómo se ha diseñado la solución**, sin entrar todavía en detalles de código.

Secciones típicas:

1. Arquitectura de alto nivel

- Componentes principales.
- Cómo se comunican.
- Tecnologías elegidas (a alto nivel) y justificación.

2. Modelo de datos

- Esquemas de base de datos.
- Diagramas de clases / entidades.

3. Diseño de la interacción y la interfaz (si aplica)

- Flujos de usuario.
- Wireframes, prototipos.
- Decisiones de diseño importantes (por ejemplo, por accesibilidad).

4. Diseño de API o servicios (si aplica)

- Endpoints principales.
- Formato de datos (JSON, etc.).

Objetivo: que alguien con conocimientos técnicos pueda entender la forma general de tu solución sin mirar el código.

4.9. Implementación

En este capítulo cuentas **cómo se ha materializado el diseño en código**.

Secciones típicas:

- Descripción de los módulos más importantes:
 - Qué hacen.
 - Cómo se relacionan.
- Tecnologías concretas utilizadas:
 - Lenguajes, frameworks, librerías clave.
- Decisiones técnicas relevantes:
 - P.ej., por qué se eligió un framework frente a otro.
- Problemas encontrados y cómo se han resuelto.

No es necesario mostrar todo el código, pero sí:

- Fragmentos representativos.
 - Ejemplos de uso.
 - Referencias al repositorio en GitHub cuando sea relevante.
-

4.10. Evaluación / experimentación

Este capítulo responde a:

¿Qué tan bien funciona lo que has hecho?

Según el tipo de proyecto:

- **Aplicación de usuario final:**
 - Pruebas de usabilidad.
 - Pruebas de rendimiento, tiempos de respuesta, etc.
- **Proyecto de ciencia de datos / IA:**
 - Diseño de experimentos.
 - Conjuntos de datos utilizados.
 - Métricas y resultados (precisión, recall, F1, etc.).
- **Proyecto de infraestructura / sistemas:**
 - Pruebas de carga, mediciones de latencia, disponibilidad, etc.

Recomendaciones:

- Describe la metodología de evaluación:
 - Cómo se han diseñado los tests.
 - Qué métricas se han elegido y por qué.

- Presenta resultados de forma clara:
 - Tablas y gráficos.
 - Comentarios y análisis crítico (no solo “estos son los números”).
-

4.11. Conclusiones y trabajo futuro

En este capítulo cierras el proyecto:

- **Resumen de lo conseguido:**
 - Repasar objetivos y explicar en qué grado se han cumplido.
 - **Principales aportaciones:**
 - Técnicas, de diseño, de integración, etc.
 - **Limitaciones:**
 - Qué no se ha podido hacer y por qué.
 - **Líneas de trabajo futuro:**
 - Qué caminos razonables quedan abiertos:
 - Mejora de la solución.
 - Ampliación a otros contextos.
 - Integración con sistemas externos, etc.
-

4.12. Referencias

Se recomienda:

- Usar un gestor de referencias (BibTeX, Zotero, etc.).
- Mantener un estilo homogéneo (APA, IEEE, etc., según la normativa del centro).

Asegúrate de:

- Citar correctamente todo trabajo previo que utilices.
 - No abusar de enlaces sin contexto: mejor referencias formales cuando sea posible.
-

4.13. Anexos

En los anexos puedes incluir:

- Diagramas detallados.
- Especificaciones técnicas extensas.
- Cuestionarios, guiones de entrevistas.
- Listados largos de código (si son imprescindibles).
- Manuales de usuario, etc.

La idea es que la memoria principal no quede saturada, pero la información necesaria para reproducir resultados o entender detalles esté disponible.

4.14. Recomendaciones prácticas

- **No dejes la memoria para el final:**
 - Ve escribiendo en paralelo.
 - Puedes usar issues/tareas específicas del tipo “Escribir sección X”.
- **Escribe pensando en alguien que no estuvo contigo** durante el TFG/TFM:
 - Que pueda entender qué has hecho, por qué y cómo.
- **Coherencia interna:**
 - Asegúrate de que objetivos, metodología, resultados y conclusiones encajan entre sí.

La memoria es parte esencial de la evaluación: no solo importa el producto final, sino también **cómo cuentas el proceso y los resultados**.

4.15. Recursos complementarios recomendados

Además de este handbook, es muy recomendable que consultes otros recursos ya existentes en la UGR y en la ETSIIT, especialmente para profundizar en la parte **formal** de la memoria:

4.15.1. Libro sobre TFG/TFM en la ETSIIT

- Tal vez ya conozcas el libro *Cómo escribir la memoria de tu TFG del Grado en Ingeniería Informática y presentarlo sin morir en el intento* publicado en abierto por profesores compañeros de la ETSIIT.

Este libro ofrece:

- Una visión general de qué es un TFG/TFM en el contexto de la ETSIIT.
- Recomendaciones sobre estructura, estilo y redacción académica.
- Pautas sobre evaluación y aspectos formales.

4.15.2. Plantilla de TFG de JJ Merelo (ETSIIT)

- Repositorio: <https://github.com/JJ/plantilla-TFG-ETSIIT>

Aunque hace bastante tiempo que no se actualiza, es especialmente útil si:

- Quieres escribir la memoria en **LaTeX**.
- Prefieres partir de una plantilla ya adaptada a la ETSIIT.
- Te interesa ver un ejemplo de automatización del proceso de compilación.

Puedes usar:

- Este documento como guía sobre **qué** debe contener la memoria y cómo organizarla.
- El libro de la UGR y la plantilla de JJ como referencia para el **formato**, el estilo y buenas prácticas de redacción académica.